

REPUBLIQUE DU SENEGAL



Un peuple - Un but - Une foi

Ministère de l'Enseignement Supérieur, de la Recherche et de l'Innovation

Direction de l'Enseignement Supérieur Privé

Institut Supérieur d'Informatique

ISI

**Rapport de stage pour l'obtention de la licence professionnelle en Génie
Logiciel**

**Conception et développement d'une application
web de gestion de bibliothèque**

Présenté et soutenu par :

M. El Hadji Serigne Fall

Sous la direction :

**M. Matar THIOYE
Spécialité : Développeur Sénior
et Formateur en informatique**

Année Académique : 2025-2026

REPUBLIQUE DU SENEGAL



Un peuple - Un but - Une foi

Ministère de l'Enseignement Supérieur, de la Recherche et de l'Innovation

Direction de l'Enseignement Supérieur Privé

Institut Supérieur d'Informatique

ISI

**Rapport de stage pour l'obtention de la licence professionnelle en Génie
Logiciel**

**Conception et développement d'une application
web de gestion de bibliothèque**

Présenté et soutenu par :

M. El Hadji Serigne Fall

Sous la direction :

**M. Matar THIOYE
Spécialité : Développeur Sénior
et Formateur en informatique**

Année Académique : 2025-2026

A la mémoire

de tous les musulmans et musulmanes qui ont quitté ce monde.

Que leurs âmes reposent en paix, que la miséricorde divine les enveloppe, et que leur souvenir, à l'image de l'œuvre de Serigne Touba Khadimou Rassoul, continue d'inspirer et de guider ceux qui restent.

Dédicace

À mes chers parents,

En témoignage de ma profonde reconnaissance et de mon immense gratitude pour votre amour inconditionnel, votre patience, votre soutien indéfectible, vos précieux conseils et tous les sacrifices que vous avez consentis pour mon éducation et mon épanouissement.

Aucune parole, aucune dédicace ne saurait exprimer pleinement le respect, l'admiration et l'amour profond que je vous porte. Votre présence, vos encouragements et votre confiance ont toujours été pour moi une source de force et de motivation.

Que Dieu vous accorde une longue vie, une excellente santé et un bonheur durable, afin que je puisse, à mon tour, vous rendre fiers et vous témoigner toute la reconnaissance que vous méritez.

Avec tout mon amour et mon profond respect. ♥

À mes chers parents.

Remerciements

Alkhamdoulilah Hanla kouli haal Dieu merci

Avant toute chose, je rends grâce à **Allah**, le Tout-Puissant, pour Sa miséricorde, Sa protection et la force qu'Il m'a accordée tout au long de mon parcours académique. Sans Sa volonté, ce travail n'aurait pu être réalisé.

Je tiens ensuite à exprimer ma plus profonde gratitude à **mes chers parents**. Votre amour inconditionnel, votre soutien constant, vos sacrifices et vos prières ont été la source de ma motivation et de ma persévérance. Merci de m'avoir toujours encouragé à poursuivre mes études, à croire en mes capacités et à ne jamais renoncer face aux difficultés. Ce travail est aussi le fruit de vos efforts.

J'adresse également mes sincères remerciements à **toute ma famille**, en particulier à **mes sœurs**, pour leur affection, leur patience, leurs encouragements et leur présence à mes côtés tout au long de ce parcours. Votre soutien moral a été d'une valeur inestimable.

Mes remerciements vont également à **l'ensemble du corps professoral de l'Institut Supérieur d'Informatique (ISI)** pour la qualité des enseignements dispensés, leur disponibilité et leur engagement dans notre formation. Grâce à leur accompagnement, j'ai pu acquérir les connaissances et les compétences nécessaires à la réalisation de ce projet.

Je tiens à adresser une reconnaissance toute particulière à **mon professeur encadrant, Monsieur Matar Thioye**, pour son accompagnement, sa disponibilité, ses précieux conseils, sa patience et ses orientations tout au long de la réalisation de ce travail. Son expertise et son encadrement ont été d'une aide précieuse et ont largement contribué à l'aboutissement de ce projet.

Enfin, j'exprime toute ma gratitude à **mes amis, mes camarades de classe ainsi qu'à mon groupe d'amis**, qui m'ont soutenu de près ou de loin durant cette aventure. Merci pour votre amitié, vos encouragements, vos échanges et les nombreux moments de partage qui ont rendu ce parcours plus agréable et enrichissant.

À toutes les personnes qui ont contribué, directement ou indirectement, à la réalisation de ce travail, je présente mes sincères remerciements.

Avant-propos

Créé en 1988, l'Institut Supérieur d'Informatique (ISI) est un établissement d'enseignement supérieur privé qui propose des formations dans les domaines de l'informatique et de la gestion. Il offre diverses spécialités, notamment : l'informatique de gestion, la comptabilité, l'infographie, les réseaux informatiques, les télécommunications, ainsi que le génie logiciel. L'institut délivre plusieurs diplômes tels que le Brevet de Technicien Supérieur (BTS), le Diplôme de Technicien Supérieur (DTS), la Licence et le Master Professionnels, dont la plupart sont reconnus par le CAMES et l'ANAQSUP.

C'est dans ce cadre que nous avons élaboré ce document, dont le sujet est : **Conception et développement d'une application web de gestion de bibliothèque.**

Ce projet académique a pour objectif la conception et le développement d'une application web permettant de moderniser la gestion des bibliothèques. L'application couvre la gestion des ouvrages, des adhérents, des prêts et retours, ainsi que le suivi des pénalités, offrant ainsi une solution numérique adaptée aux bibliothèques universitaires, publiques, scolaires et privées.

Ce document constitue notre premier travail de recherche académique, et à ce titre, nous sollicitons de la part du jury beaucoup d'indulgence pour l'évaluation de ce projet.

Sommaire

A la mémoire de	I
Dédicace	II
Remerciements	III
Avant-propos	IV
Glossaire	VI
Liste des figures	VII
Résumé	VIII
Abstract	IX
PREMIER CHAPITRE I – INTRODUCTION GÉNÉRALE	1
1.1 Présentation du domaine des bibliothèques	2
1.2 Contexte	3
1.3 Problématique	4
1.4 Objectifs	5
DEUXIEME CHAPITRE II – Etude et Réalisation	6
2.1 Gestion de projet agile	7
2.2 Spécification fonctionnelle	8
2.3 Analyse et conception	11
2.4 Réalisation	14
TROISIEME CHAPITRE III – Bilan	25
3.1 Bilan	25
3.2 Intérêts personnels	26
3.3 Apport du projet pour les bibliothèques	28

BIBLIOGRAPHIE	i
WEBOGRAPHIE	ii
Table des matières	iii

Glossaire

Tableau 1 : Sigles et Abréviations

SIGLES & ABRÉVIATIONS	SIGNIFICATIONS	SIGLES & ABRÉVIATIONS	SIGNIFICATIONS
API	Interface de programmation d'application	BD	Base de données
CSS	Cascading Style Sheets	FCFA	Franç CFA (monnaie)
HTML	HyperText Markup Language	ISBN	International Standard Book Number
ISI	Institut Supérieur d'Informatique	JS	JavaScript
MVC	Modèle Vue Contrôleur	MySQL	My Structured Query Language
PDF	Portable Document Format	PHP	Hypertext Preprocessor
QR	Quick Response	SGBD	Système de Gestion de Base de Données
SQL	Structured Query Language	UML	Unified Modeling Language

Liste des figures

Figure 1 : Méthodes Agiles	7
Figure 2 : Tableau de bord Trello	8
Figure 3 : Langage UML	11
Figure 4 : Diagramme de contexte	11
Figure 5 : Diagramme de cas d'utilisation générale	12
Figure 6 : Diagramme de cas d'utilisation – Acteur Bibliothécaire	13
Figure 7 : Diagramme de cas d'utilisation – Acteur Administrateur	13
Figure 8 : Diagramme de cas d'utilisation – Acteur Adhérent	13
Figure 9 : Diagramme de classe	14
Figure 10 : Architecture de l'application	15
Figure 11 : Technologies et langages utilisés	15
Figure 12 : Architecture MVC de Laravel	16
Figure 13 : Page d'accueil	19
Figure 14 : Page de connexion	20
Figure 15 : Tableau de bord administrateur	20
Figure 16 : Formulaire d'ajout d'un ouvrage	20
Figure 17 : Gestion des catégories	21
Figure 18 : Gestion des exemplaires et codes-barres	21
Figure 19 : Liste des ouvrages	22
Figure 20 : Gestion des adhérents	22
Figure 21 : Carte adhérent PDF	23
Figure 22 : Gestion des prêts en cours	23
Figure 23 : Gestion des pénalités	24
Figure 24 : Maquette Figma – Page de connexion	25
Figure 25 : Maquette Figma – Tableau de bord administrateur	25

Liste des tableaux

Tableau 1 : Sigles et Abréviations	VI
Tableau 2 : Besoins fonctionnels par module	11

Résumé

La gestion des ressources documentaires constitue un défi majeur pour de nombreuses institutions : universités, lycées, bibliothèques publiques et centres de documentation privés. La multiplication des fonds documentaires, combinée à des processus encore largement manuels, génère des inefficacités importantes : pertes de données, erreurs de suivi des prêts, difficulté d'accès aux informations et insatisfaction croissante des usagers.

Face à ces constats, ce projet académique propose la conception et le développement d'une application web de gestion de bibliothèque. La solution permettra aux usagers de consulter un catalogue en ligne, d'emprunter des ouvrages et de suivre l'évolution de leurs prêts. Elle facilitera également le travail des bibliothécaires grâce à une base de données centralisée et des outils de recherche performants. Les administrateurs disposeront d'un tableau de bord complet pour superviser l'ensemble des activités.

En résumé, cette application intègre sept modules fonctionnels : la gestion des ouvrages (catalogage, exemplaires, codes-barres), la gestion des adhérents (inscription, types, cartes QR code), la gestion des prêts et retours, le calcul automatique des pénalités en FCFA, un espace adhérent en ligne, des tableaux de bord statistiques avec export Excel et CSV, ainsi qu'un module d'administration des rôles et permissions. Cette solution est conçue pour s'adapter à tout type de bibliothèque.

Abstract

Managing documentary resources is a significant challenge for many institutions, including universities, public libraries, schools, and private documentation centers. The combination of growing collections and largely manual processes leads to major inefficiencies: data loss, loan tracking errors, limited access to information, and increasing user dissatisfaction.

In response to these issues, this academic project proposes the design and development of a web-based library management application. The solution will allow users to browse an online catalog, borrow books, and track their loans in real time. It will also support librarians through a centralized database and efficient search tools, while providing administrators with a comprehensive dashboard to oversee all library activities.

In summary, this application covers seven functional modules: book management (cataloging, copies, barcodes), member management (registration, types, QR code cards), loan and return management, automatic penalty calculation in FCFA, an online member portal, statistical dashboards with Excel and CSV export, and a role and permission administration module. The solution is designed to be adaptable to any type of library, whether public, private, university, or school-based.

PREMIER CHAPITRE

INTRODUCTION GÉNÉRALE

1.1 Présentation du domaine des bibliothèques

Les bibliothèques jouent un rôle fondamental dans la diffusion du savoir et l'accès à la culture. Qu'elles soient universitaires, publiques, scolaires ou privées, elles constituent des espaces essentiels pour les étudiants, les enseignants, les chercheurs et le grand public. Leur mission principale est de collecter, organiser, conserver et mettre à disposition des ressources documentaires variées : livres, périodiques, ressources numériques et archives.

Objectifs

- Centraliser les ressources : Regrouper l'ensemble des informations relatives aux ouvrages, aux adhérents et aux prêts dans un système unique et accessible.
- Automatiser les processus : Réduire les tâches manuelles grâce à la numérisation des prêts, retours, pénalités et notifications, afin d'améliorer la productivité du personnel.
- Renforcer la traçabilité : Assurer un suivi rigoureux des ouvrages et des adhérents, avec des mécanismes de contrôle et des journaux d'activité détaillés.
- Améliorer l'expérience utilisateur : Offrir aux adhérents un espace en ligne pour consulter le catalogue, suivre leurs emprunts et effectuer des demandes de prolongation sans avoir à se déplacer.

Fonctionnalités

- Catalogue en ligne : L'application met à disposition un catalogue consultable par tous les usagers, avec des fonctionnalités de recherche avancée par titre, auteur, ISBN et catégorie.
- Espace de gestion : Les bibliothécaires disposent d'une interface dédiée pour gérer les ouvrages, les adhérents, les prêts et les pénalités de manière efficace et centralisée.
- Suivi en temps réel : Les adhérents peuvent consulter l'état de leurs emprunts en ligne, recevoir des notifications d'échéance et prolonger leurs prêts depuis leur espace personnel.

Impact attendu

- Réduction des délais : La numérisation des processus vise à réduire significativement le temps de traitement des prêts, des retours et des inscriptions.
- Accessibilité élargie : Garantir un accès équitable aux ressources documentaires pour tous les usagers, quel que soit leur profil (étudiant, enseignant, chercheur, membre externe).
- Satisfaction des usagers : Améliorer la qualité des services proposés grâce à une interface intuitive, des notifications automatiques et une disponibilité en ligne permanente.

1.2 Contexte

À l'ère du numérique, de nombreux secteurs ont amorcé leur transition vers des solutions informatiques permettant de rationaliser leurs processus. Pourtant, les bibliothèques — qu'elles soient universitaires, publiques, scolaires ou privées — peinent encore à franchir ce cap. Dans beaucoup d'établissements, la gestion des ouvrages, des adhérents et des prêts repose encore sur des registres papier ou des tableurs, ce qui engendre de nombreuses difficultés.

Actuellement, les processus de gestion des bibliothèques présentent plusieurs problèmes :

- Difficulté de suivi des ouvrages : Une fois l'ouvrage emprunté, le suivi de sa localisation et de sa disponibilité est difficile, avec peu ou pas de moyens pour les usagers de savoir où en est leur demande.
- Manque de centralisation : Les informations concernant les ouvrages, les adhérents et les prêts peuvent être dispersées entre différents fichiers et registres, rendant difficile la coordination et le suivi.
- Temps de traitement prolongé : Le délai entre la demande d'emprunt et l'enregistrement effectif peut être long, ce qui pose des problèmes supplémentaires aux usagers ayant besoin d'ouvrages rapidement.
- Risque d'erreurs humaines : Le traitement manuel des prêts et des retours augmente le risque d'erreurs humaines, telles que l'oubli d'enregistrement d'un retour ou la perte de dossiers.
- Inaccessibilité des services : Les horaires d'ouverture de la bibliothèque peuvent limiter l'accès des usagers à certains services (prolongation, consultation), en particulier pour ceux qui ont des horaires de travail contraignants.

- Sécurité des données personnelles : Les méthodes actuelles de gestion peuvent ne pas offrir un niveau de sécurité adéquat pour protéger les données personnelles des adhérents.
- Coût élevé pour les organismes : Les bibliothèques doivent souvent allouer des ressources considérables pour gérer manuellement les prêts et les retours, ce qui peut être coûteux et inefficace.
- Gestion complexe des pénalités : Le calcul manuel des amendes de retard est source d'erreurs et de conflits avec les usagers.
- Absence de statistiques : L'absence d'outils d'analyse rend difficile la prise de décisions stratégiques pour l'achat de nouveaux ouvrages.

1.3 Problématique

La problématique centrale de ce projet est la suivante : comment concevoir une application web capable de moderniser et d'automatiser la gestion d'une bibliothèque, en répondant aux besoins fonctionnels des bibliothécaires, des administrateurs et des adhérents ? Cette question soulève plusieurs axes de réflexion :

- Digitalisation de la gestion des prêts : Pour moderniser ce processus, il serait avantageux de développer une plateforme en ligne dédiée à la gestion des ouvrages, des adhérents et des prêts. Cette solution permettrait aux usagers de consulter le catalogue, d'emprunter des ouvrages et de suivre leurs prêts de manière simple et rapide via l'application.
- Amélioration de la communication et de la coordination : La communication efficace entre les usagers et les bibliothécaires est cruciale pour le succès de ce processus. Une plateforme numérique pourrait intégrer des fonctionnalités de notification par email pour informer les usagers de l'échéance de leurs prêts ou de la disponibilité d'ouvrages réservés.
- Système de suivi en temps réel : Un autre point essentiel est l'implémentation d'un système de suivi en temps réel pour les prêts en cours. Une plateforme centralisée offrirait aux usagers la possibilité de suivre l'état de leurs emprunts, de recevoir des mises à jour instantanées et d'être informés des pénalités éventuelles. Cela diminuerait significativement la nécessité de se déplacer à la bibliothèque pour des vérifications.

- **Sécurisation des données :** La protection des informations personnelles des adhérents est une préoccupation majeure. La plateforme intégrerait des technologies de cryptage avancées et des protocoles de sécurité stricts afin de garantir la sécurité des données sensibles des utilisateurs.
- **Automatisation des pénalités :** Le calcul automatique des amendes de retard en FCFA, selon le type d'adhérent, permettrait d'éliminer les erreurs humaines et les conflits avec les usagers.

La mise en œuvre de solutions informatiques telles qu'une plateforme web pour la gestion des bibliothèques, un système de suivi en temps réel, des mesures de sécurité des données, l'automatisation des processus, l'amélioration de la communication, et une interface dédiée pour les usagers pourrait considérablement améliorer l'efficacité, la sécurité et la satisfaction des usagers.

1.4 Objectifs

- **Gestion des ouvrages :** Permettre aux bibliothécaires d'ajouter, modifier, archiver et supprimer des ouvrages. Gérer les catégories et les sous-catégories d'ouvrages. Gérer les exemplaires de chaque ouvrage avec leur état. Générer automatiquement des codes-barres pour chaque exemplaire.
- **Gestion des adhérents :** Permettre aux bibliothécaires d'inscrire de nouveaux adhérents et de gérer le renouvellement des adhésions. Classer les adhérents par types (étudiant, enseignant, chercheur, externe). Générer des cartes d'adhérents en PDF avec photo et QR code intégré.
- **Gestion des prêts et retours :** Enregistrer les prêts et retours d'ouvrages via scan de code-barres. Gérer les prolongations de prêt en ligne depuis l'espace adhérent. Envoyer des notifications par email pour les rappels d'échéance et les retards.
- **Gestion des pénalités :** Calculer automatiquement les amendes en FCFA selon le barème configuré. Enregistrer les paiements partiels ou totaux des pénalités. Bloquer les nouveaux prêts pour les adhérents ayant des pénalités impayées.
- **Espace adhérent :** Permettre aux usagers de consulter le catalogue public et de rechercher des ouvrages. Permettre aux usagers de visualiser leurs prêts en cours et leur historique. Permettre la prolongation de prêt en ligne (si éligible).

- Tableaux de bord et statistiques : Afficher des graphiques interactifs sur les emprunts, les ouvrages les plus consultés. Exporter les données en Excel et CSV. Générer des rapports périodiques (hebdomadaires, mensuels, annuels).
- Administration : Gérer les utilisateurs du système (bibliothécaires, administrateurs). Définir les rôles et les permissions d'accès. Consulter les journaux d'activité (logs) du système.

2.1 Gestion de projet agile

La gestion de projet Agile a pris d'assaut le monde des affaires ces deux dernières décennies et, plus récemment, Scrum domine le paysage. Scrum est un ensemble de pratiques, de rôles, d'événements, d'artefacts et de règles conçus pour guider l'équipe dans l'exécution du projet.

C'est précisément le framework Scrum qui a été retenu et appliqué pour la conduite de ce projet. Le développement du projet BiblioSmart a été organisé en plusieurs sprints d'une à deux semaines, chacun correspondant à un ensemble cohérent de fonctionnalités à livrer (par exemple : un sprint dédié à la gestion des ouvrages, un autre à la gestion des adhérents, un autre aux prêts et pénalités). Chaque sprint a suivi le cycle Scrum classique, adapté au contexte d'un projet académique mené en autonomie : une phase de planification (sprint planning) où les tâches du sprint étaient définies et priorisées dans le backlog produit, un suivi quotidien de l'avancement des tâches (équivalent du daily stand-up, ici formalisé par la mise à jour régulière du tableau Trello), puis une revue de sprint (sprint review) permettant de valider les fonctionnalités développées par rapport aux besoins fonctionnels définis en amont. Les rôles Scrum ont été adaptés à un contexte de développement individuel : l'étudiant a assumé conjointement les responsabilités de Product Owner (priorisation des besoins fonctionnels, définition du backlog) et de développeur (implémentation), le rôle de Scrum Master (facilitation du processus, levée des blocages) étant partagé avec l'encadrant du projet lors des points de suivi. Les artefacts Scrum ont été matérialisés de la façon suivante : le backlog produit et le backlog de sprint ont été gérés via l'outil Trello (voir 2.1.1), sous forme de tableaux Kanban avec des listes "À faire", "En cours" et "Terminé", et l'incrément (le logiciel fonctionnel livré à la fin de chaque sprint) correspondait à chaque nouvelle fonctionnalité intégrée et testée dans l'application. Cette approche itérative et incrémentale a permis d'obtenir régulièrement un retour sur l'avancement du projet, de réajuster les priorités en fonction des difficultés rencontrées, et de livrer progressivement un produit de plus en plus complet plutôt que d'attendre la fin du projet pour disposer d'une version fonctionnelle.

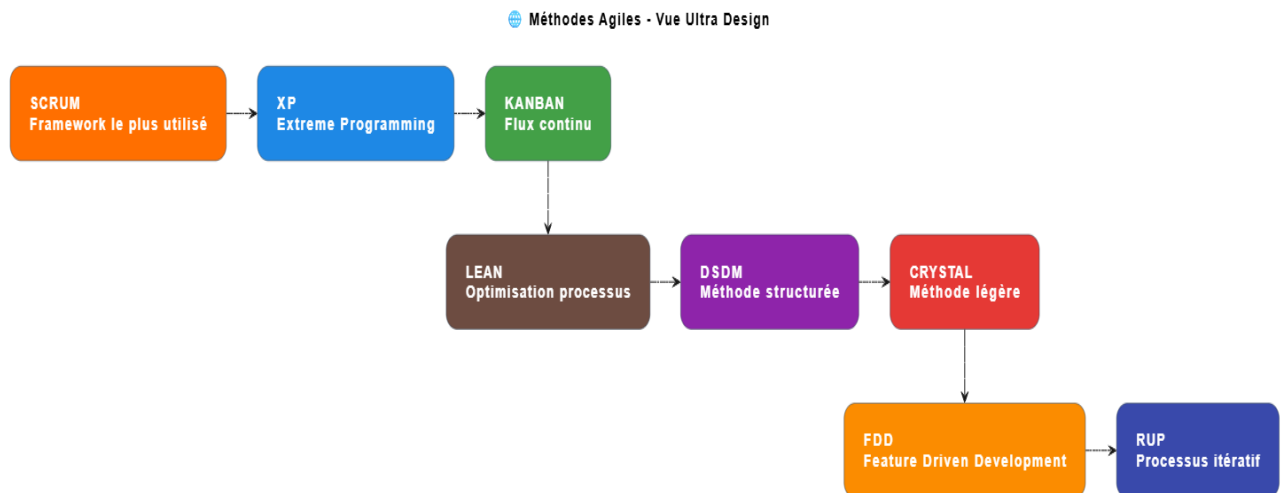


Figure 1 : Méthodes Agiles

2.1.1 Trello

Pour la gestion de notre projet, nous avons choisi Trello comme outil principal. Trello est un outil de gestion de projet basé sur des tableaux Kanban, qui permet de visualiser et de suivre l'avancement des tâches de manière simple et efficace.

- **Tableaux** : Les tableaux dans Trello représentent les différents aspects ou phases de notre projet. Chaque tableau est un espace dédié où nous organisons et visualisons les tâches à accomplir pour une partie spécifique du projet.
- **Listes** : Au sein de chaque tableau, les listes nous permettent de structurer les étapes du projet. Par exemple, les listes peuvent représenter des étapes telles que "À faire", "En cours", et "Terminé", facilitant ainsi le suivi de l'avancement des tâches.
- **Cartes** : Les cartes sont les éléments de base qui constituent les tâches spécifiques à réaliser. Chaque carte peut inclure des descriptions, des membres responsables, des dates d'échéance, des checklists, et des pièces jointes, ce qui permet une gestion détaillée et organisée des tâches.

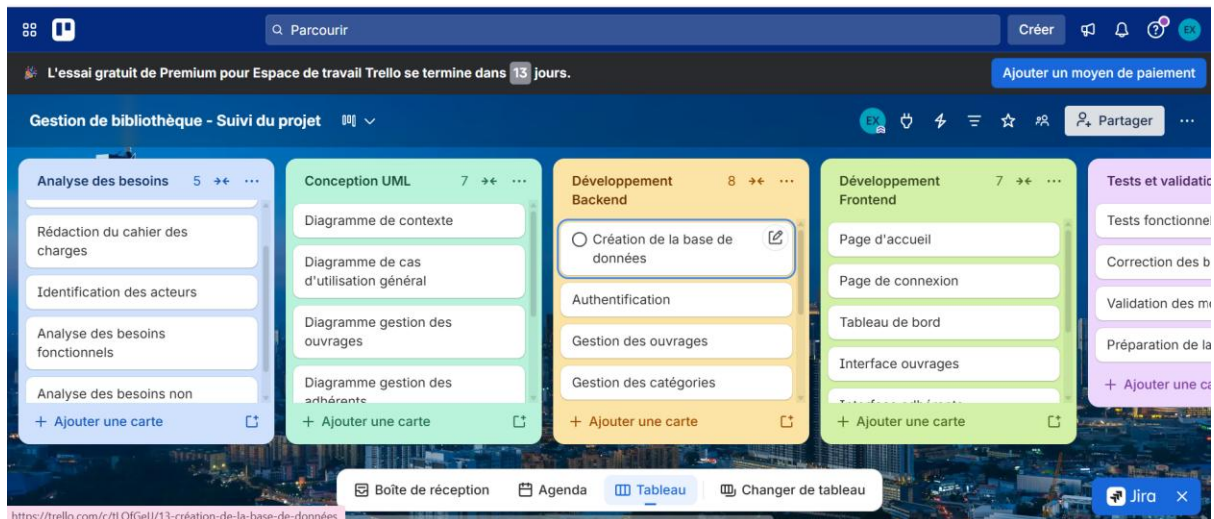


Figure 2 : Tableau de bord Trello

2.2 Spécification fonctionnelle

2.2.1 Les besoins fonctionnels

L'application de gestion de bibliothèque est conçue pour faciliter la gestion des ouvrages, des adhérents, des prêts et des pénalités. Elle doit répondre aux besoins suivants :

Gestion des ouvrages

- Formulaire pour entrer les détails d'un ouvrage (titre, auteur, ISBN, catégorie, description, couverture).
- Gestion des exemplaires avec génération automatique de codes-barres.
- Recherche avancée par titre, auteur, ISBN, catégorie.

Gestion des adhérents

- Inscription des adhérents avec saisie des informations personnelles.
- Classification par types (étudiant, enseignant, chercheur, externe).
- Génération de cartes d'adhérents en PDF avec QR code.
- Gestion des renouvellements d'adhésion.

Gestion des prêts et retours

- Enregistrement des prêts via scan de code-barres de l'adhérent et de l'exemplaire.
- Enregistrement des retours avec mise à jour du statut de l'exemplaire.
- Prolongation de prêt en ligne (espace adhérent).
- Vérification automatique des conditions d'éligibilité (adhésion valide, quota non atteint, pas de pénalités bloquantes).

Gestion des pénalités

- Calcul automatique des amendes en FCFA selon le barème configuré.
- Enregistrement des paiements partiels ou totaux.
- Blocage automatique des nouveaux prêts pour les adhérents avec pénalités impayées.

Espace adhérent

- Consultation du catalogue public.
- Visualisation des prêts en cours et de l'historique.
- Demande de prolongation en ligne.
- Mise à jour des informations personnelles.

Tableau de bord administrateur

- Gestion des utilisateurs (bibliothécaires, administrateurs).
- Statistiques et rapports sur les prêts, les ouvrages et les adhérents.
- Export des données en Excel et CSV.
- Consultation des journaux d'activité (logs).

Tableau 2 : Besoins fonctionnels par module

Module	Fonctionnalités
Gestion des ouvrages	Ajout, modification, archivage et suppression d'ouvrages. Gestion des catégories et sous-catégories. Gestion des exemplaires avec codes-barres automatiques.
Gestion des adhérents	Inscription des adhérents par types. Génération de cartes PDF avec QR code. Gestion des renouvellements d'adhésion.
Gestion des prêts	Enregistrement via scan de codes-barres. Vérification des conditions d'éligibilité. Prolongation en ligne. Notifications email.
Gestion des pénalités	Calcul automatique en FCFA. Paiements partiels ou totaux. Blocage des adhérents débiteurs.
Espace adhérent	Consultation du catalogue. Visualisation des prêts. Prolongation en ligne.
Tableaux de bord	Graphiques interactifs. Export Excel/CSV. Rapports périodiques.
Administration	Gestion des utilisateurs. Définition des rôles et permissions. Consultation des logs.

2.2.2 Les besoins non-fonctionnels

- Performance : L'application doit traiter efficacement les déclarations et les recherches d'ouvrages pour garantir des réponses rapides et une expérience utilisateur fluide, même avec un grand nombre de transactions.
- Scalabilité : L'application doit pouvoir s'adapter à l'ajout de nouveaux utilisateurs et à une augmentation des données et des transactions sans affecter la performance.
- Fiabilité : L'application doit être fiable avec un minimum de temps d'arrêt et des erreurs réduites.
- Sécurité : L'application doit sécuriser les informations sensibles des utilisateurs et des ouvrages. Cela inclut des mesures de protection des données, une authentification solide, une gestion rigoureuse des accès et des autorisations.

- Disponibilité : L'application doit être disponible en tout temps pour permettre aux utilisateurs de soumettre des déclarations et de rechercher des ouvrages sans interruption.
- Convivialité : L'application doit offrir une interface utilisateur claire et facile à naviguer.
- Interopérabilité : L'application doit pouvoir s'intégrer avec d'autres systèmes si nécessaire (comme des bases de données externes ou des systèmes de notification).
- Évolutivité : L'application doit être conçue pour permettre des modifications futures, telles que l'ajout de nouvelles fonctionnalités ou l'adaptation à de nouveaux processus de gestion des bibliothèques.
- Maintenabilité : L'application doit être facile à maintenir, avec une architecture et un code bien structuré.

2.3 Analyse et conception

2.3.1 La méthode UML



La méthode UML (Unified Modeling Language), en français Langage de Modélisation Unifié, est un langage de modélisation visuel largement utilisé dans le domaine du développement logiciel. UML fournit un ensemble de notations et de diagrammes pour représenter graphiquement les différentes perspectives d'un système, notamment sa structure, son comportement, ses interactions et ses processus.

Figure 3 : Langage UML

2.3.2 Le diagramme de contexte

Le diagramme de contexte en UML (Unified Modeling Language) est un type de diagramme de modélisation utilisé pour représenter visuellement le contexte général dans lequel un système informatique ou une application interagit avec son environnement. Il définit les acteurs externes (usagers, bibliothécaires, administrateurs) et leurs interactions avec le système de gestion de bibliothèque.

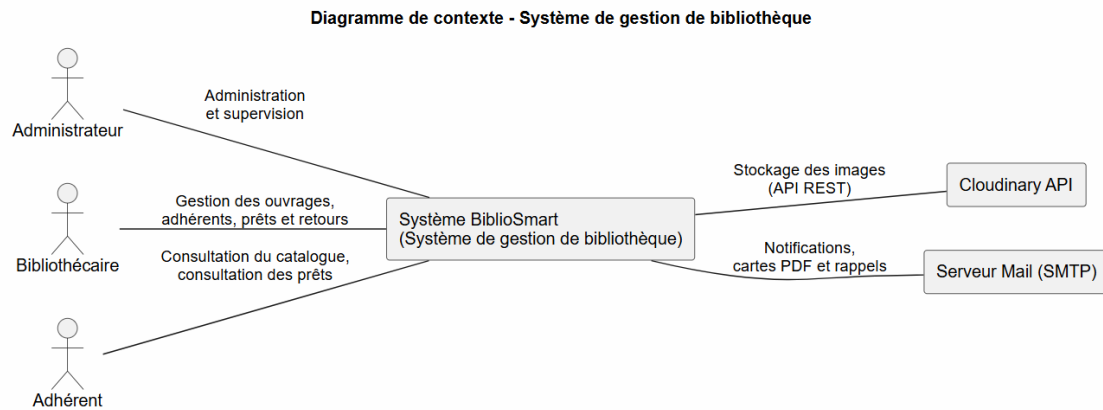


Figure 4 : Diagramme de contexte

2.3.3 Le diagramme de cas d'utilisation

Le diagramme de cas d'utilisation est un type de diagramme utilisé dans le langage de modélisation unifié (UML) pour représenter les interactions entre les acteurs (utilisateurs, systèmes externes) et un système. Il met l'accent sur les fonctionnalités ou les actions qu'un système offre à ses utilisateurs.

Formalisme diagramme de cas d'utilisation général :

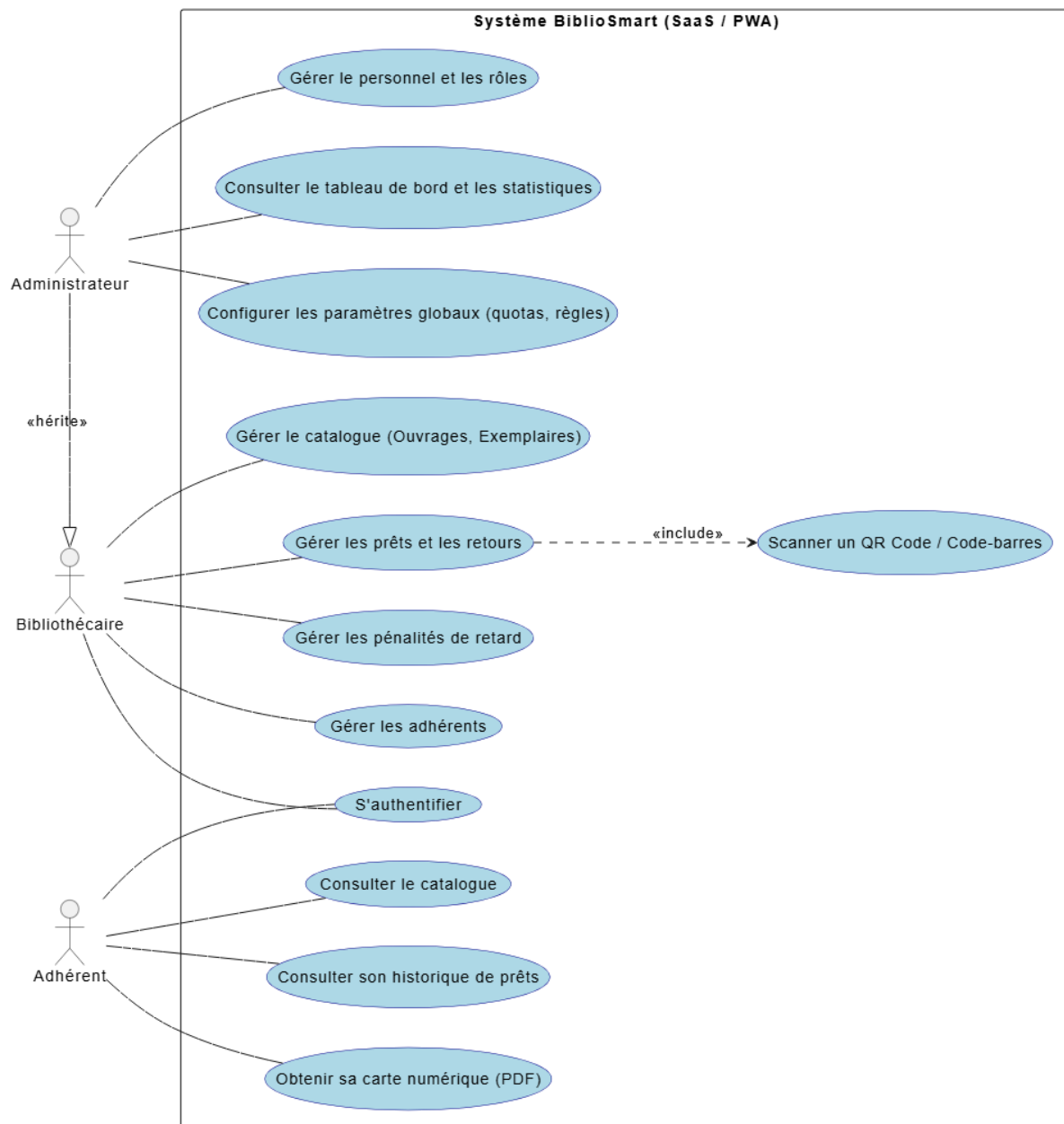


Figure 5 : Diagramme de cas d'utilisation générale

Formalisme diagramme de cas d'utilisation - Acteur Bibliothécaire :

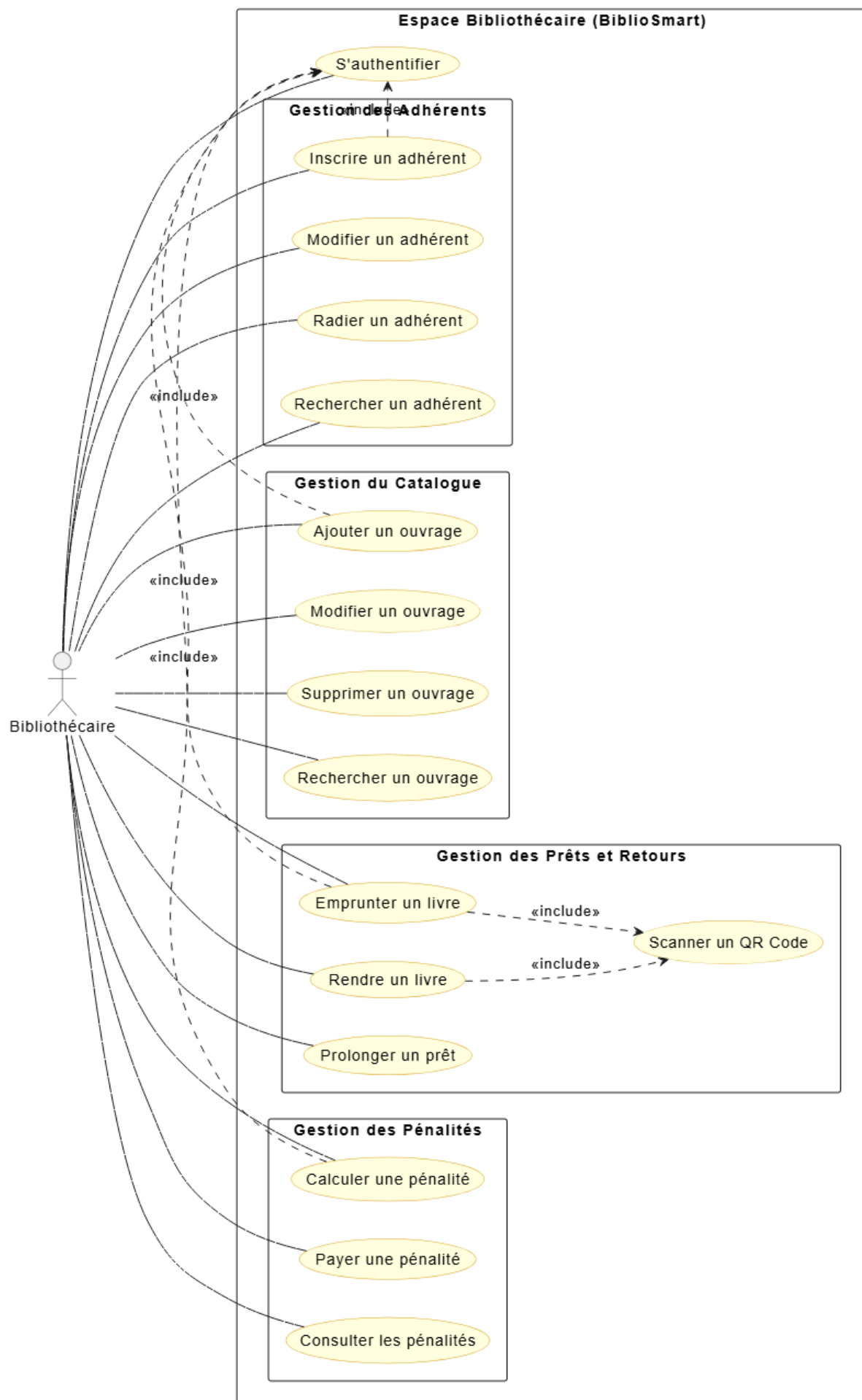


Figure 6 : Diagramme de cas d'utilisation – Acteur Bibliothécaire

Formalisme diagramme de cas d'utilisation - Acteur Administrateur :

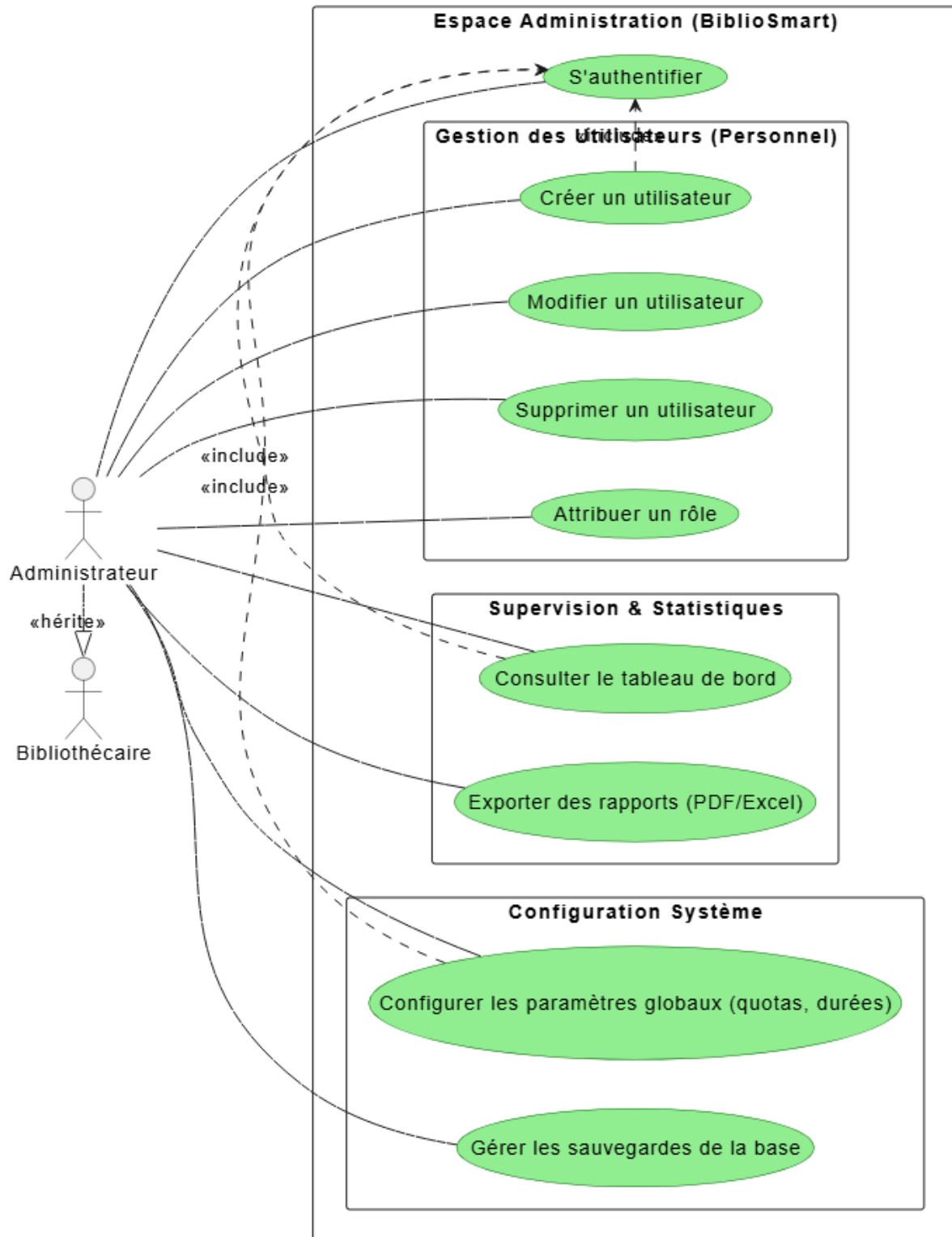


Figure 7 : Diagramme de cas d'utilisation – Acteur Administrateur

Formalisme diagramme de cas d'utilisation - Acteur Adhérent :

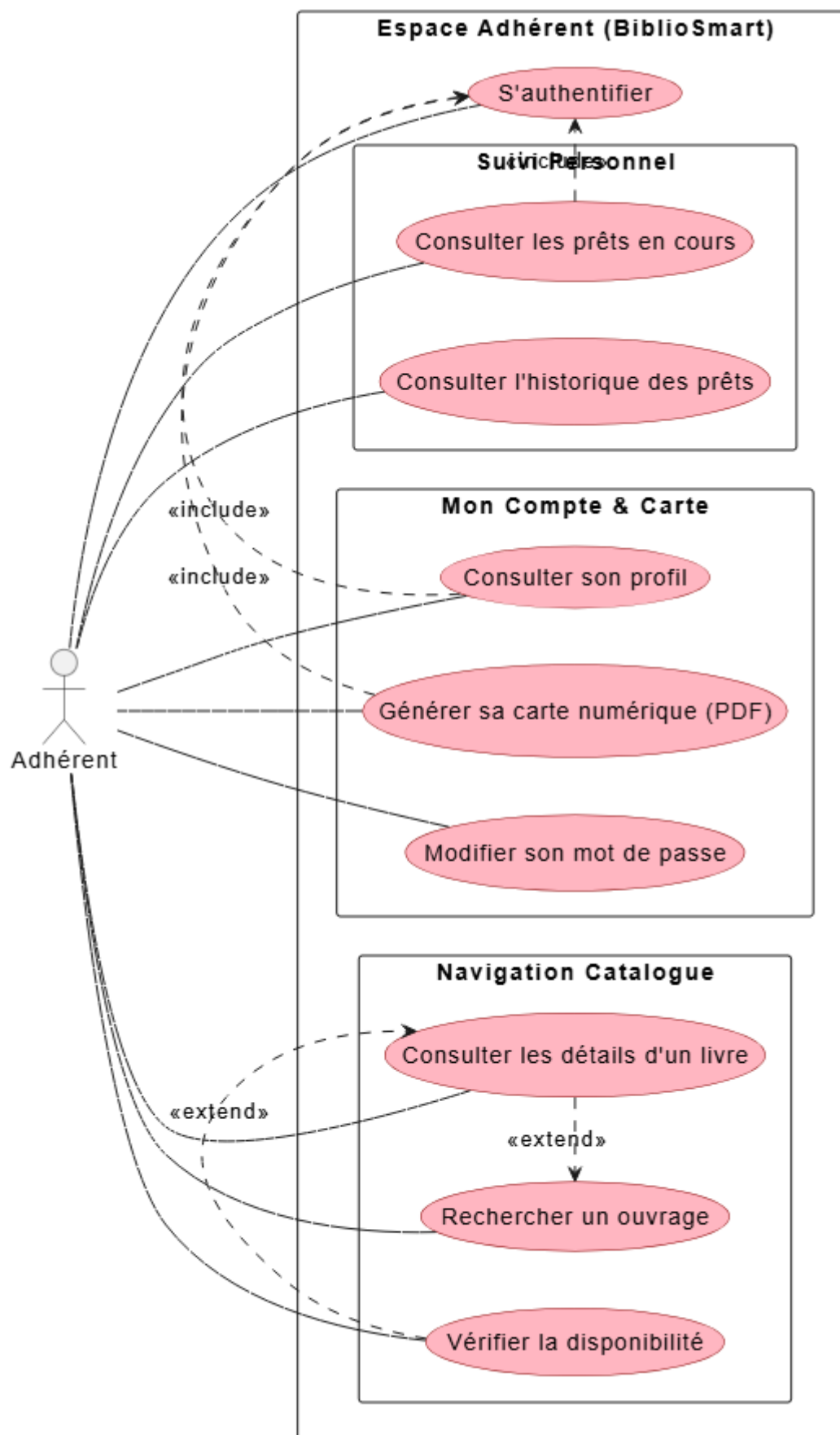


Figure 8 : Diagramme de cas d'utilisation – Acteur Adhèrent

2.3.4 Le diagramme de classe

Le diagramme de classes est un type de diagramme utilisé dans le langage de modélisation unifié (UML) permettant de représenter la structure statique d'un système logiciel. Il décrit les différentes classes du système, leurs attributs, leurs méthodes ainsi que les relations existant entre elles.

Dans notre application de gestion de bibliothèque, les principales entités identifiées sont : **Ouvrage, Exemplaire, Catégorie, Adhérent, Prêt, Retour, Pénalité et Utilisateur**. Ces classes permettent de modéliser l'ensemble des fonctionnalités du système, notamment la gestion des utilisateurs, la gestion du catalogue, ainsi que le suivi des emprunts et des retours.

2.4.1.1 Architecture

L'application repose sur une architecture client-serveur où le frontend (client) est séparé du backend (serveur). Voici les détails de l'architecture :

- Frontend : Développé avec Blade + Tailwind CSS + Alpine.js + Livewire pour une interface utilisateur réactive et dynamique.
- Backend : Basé sur le Framework Laravel pour gérer la logique métier, l'authentification, et l'accès aux données.
- API REST : Fournit une interface de communication entre le frontend et le backend.
- Base de Données : MySQL en environnement de développement local, PostgreSQL en environnement de production (Render), la transition entre les deux étant assurée par l'ORM Eloquent.

Un point d'architecture qui mérite d'être souligné est la gestion différenciée du système de gestion de base de données entre les environnements. En local, le développement s'est appuyé sur MySQL, choix pragmatique lié à sa simplicité d'installation via XAMPP et à sa large diffusion dans l'écosystème PHP. En production, en revanche, l'application est connectée à une instance PostgreSQL hébergée sur la plateforme cloud Render, réputée pour sa robustesse, sa conformité stricte aux propriétés ACID (atomicité, cohérence, isolation, durabilité) et sa gestion avancée des types de données et des contraintes d'intégrité, des qualités particulièrement adaptées à une application manipulant des données sensibles telles que les informations personnelles des adhérents ou l'historique des pénalités. Cette hétérogénéité entre les deux SGBD n'a nécessité aucune réécriture des requêtes : Laravel repose sur l'ORM Eloquent, qui constitue une couche d'abstraction entre le code applicatif et le moteur de base de données. Les modèles, les migrations et les requêtes du projet sont écrits en PHP au moyen du langage fluide d'Eloquent (query builder), et c'est le driver configuré dans le fichier .env (mysql ou pgsql) qui détermine, au moment de l'exécution, le dialecte SQL réellement généré. Ce choix illustre concrètement l'intérêt d'un ORM dans un contexte professionnel : il découple la logique métier de l'infrastructure de persistance et facilite le passage d'un environnement de développement économique vers un environnement de production plus exigeant, sans risque de régression liée à des requêtes SQL propriétaires à un moteur particulier.

Architecture 3-Tiers - Application Web Moderne 🏢

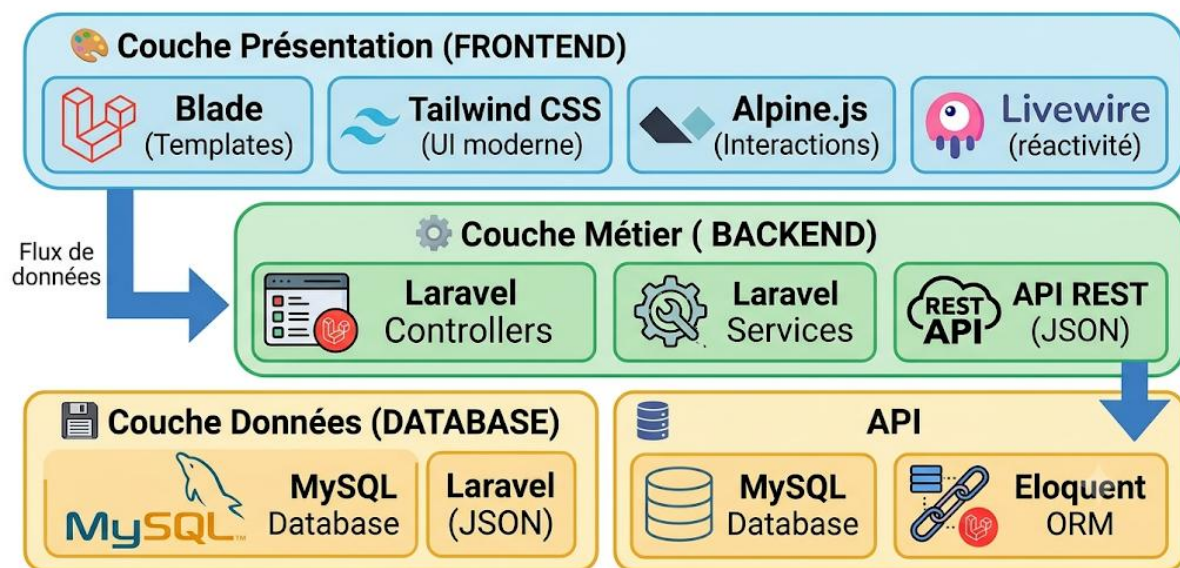


Figure 10 : Architecture de l'application

2.4.1.2 Technologies et langages

Frontend :

- Blade : Moteur de templates Laravel pour le développement d'applications web dynamiques.
- Tailwind CSS : Framework CSS utilitaire pour un design responsive et moderne.
- Alpine.js : Framework JavaScript léger pour l'interactivité côté client.
- Livewire : Composants Laravel pour des interfaces dynamiques sans rechargement de page, pivot central de la stack TALL retenue pour ce projet.

Le choix de Livewire au sein de la stack TALL n'est pas anodin : il répond à un besoin précis d'expérience utilisateur sans recourir à la complexité d'un framework JavaScript séparé. Livewire permet d'obtenir un rendu réactif proche d'une Single Page Application - mise à jour partielle du DOM, absence de rechargement de page, retours visuels instantanés - tout en conservant l'intégralité de la logique métier côté serveur, en PHP. Cette approche présente trois avantages majeurs pour un projet de cette envergure : sur le plan de la sécurité, aucune donnée sensible ni règle métier n'est exposée côté client, contrairement à une architecture React ou Vue consommant une API REST ; sur le plan de la simplicité, l'équipe de développement (réduite à une seule personne dans ce contexte académique) n'a pas à maintenir deux bases de

code distinctes (backend PHP et frontend JavaScript) ni à gérer la synchronisation d'état entre elles ; enfin, sur le plan de la productivité, Livewire permet de développer des composants interactifs (recherche en temps réel dans le catalogue, mise à jour dynamique des statuts de prêt, formulaires réactifs) directement en Blade et en PHP, ce qui a considérablement accéléré le développement de BiblioSmart.

Backend :

- Laravel (Version 11) : Framework PHP pour le développement du backend.
- PHP : Langage de script utilisé pour le développement backend.

API REST :

- JSON : Format de données pour les échanges entre frontend et backend.

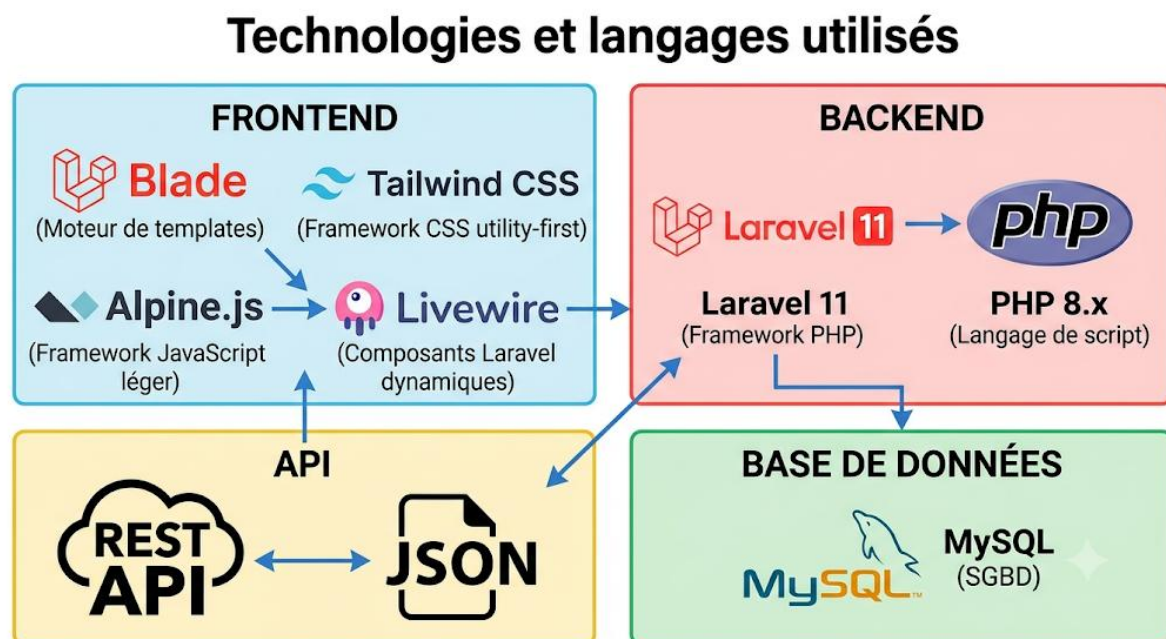


Figure 11 : Technologies et langages utilisés

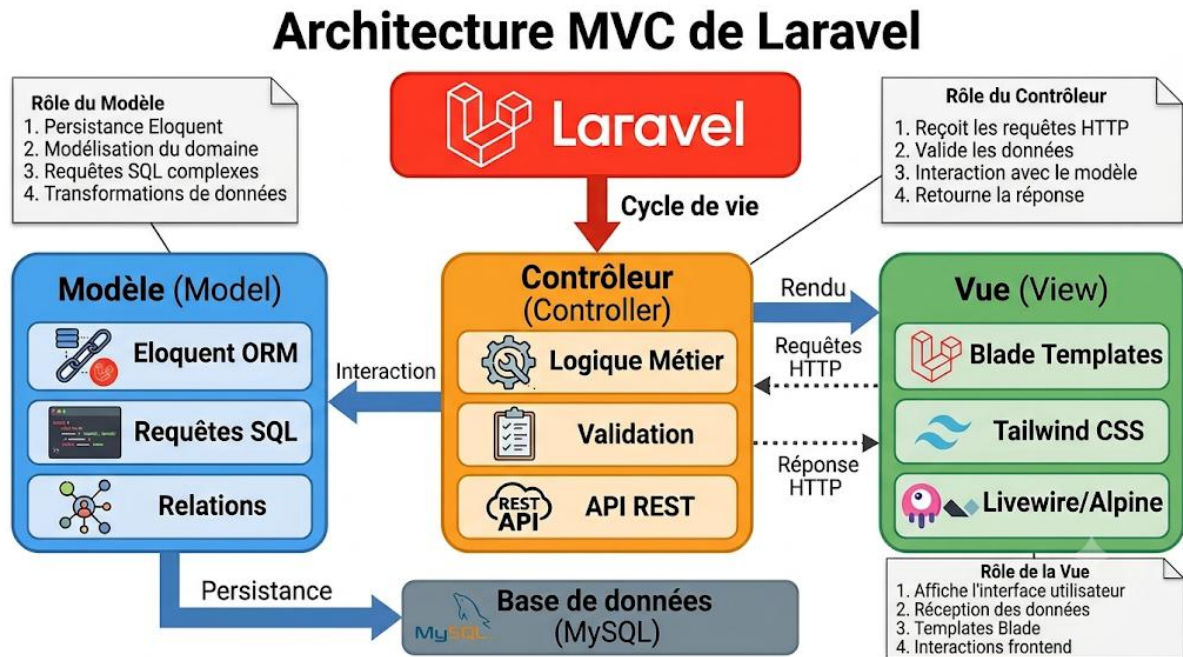


Figure 12 : Architecture MVC de Laravel

2.4.1.3 Outils

IDE :

Visual Studio Code :



Souvent abrégé en VS Code, c'est un éditeur de code source open-source développé par Microsoft. Il est largement utilisé par les développeurs pour écrire du code dans une variété de langages de programmation.

Gestion de Version :

- Git :



Système de contrôle de version décentralisé conçu pour suivre les modifications dans le code source pendant le développement logiciel. Créé par Linus Torvalds en 2005, Git permet aux développeurs de travailler simultanément sur un projet et de maintenir un historique complet des modifications.

- GitHub :



Plateforme web de gestion de code source et de collaboration basée sur Git. Elle permet aux développeurs de stocker, gérer et partager leur code, tout en facilitant la collaboration sur des projets de toutes tailles.

- **Postman :**



Outil populaire utilisé pour le développement et le test d'API. Il permet aux développeurs de tester, d'automatiser et de documenter des API facilement dans une interface utilisateur simple.

Conception et design :

- **Figma :**



Outil de prototypage et de conception d'interfaces en ligne, utilisé pour réaliser les maquettes des écrans avant développement (voir 2.4.2.8). Il permet de concevoir des interfaces interactives et de collaborer en temps réel.

Bibliothèques et services tiers :

- dompdf : Bibliothèque PHP de génération de documents PDF à partir de templates HTML/CSS, utilisée pour produire les cartes d'adhérents et les reçus de paiement (voir 2.4.2.2 et 2.4.2.4).
- simple-qrcode : Package Laravel de génération de QR codes, utilisé pour encoder l'identifiant numérique de chaque adhérent (voir 2.4.2.2).
- html5-qrcode : Librairie JavaScript permettant d'accéder à la caméra de l'appareil depuis le navigateur pour scanner les QR codes lors des emprunts et retours (voir 2.4.5.3).

- Mailtrap : Service de messagerie « sandbox » utilisé pour tester les emails transactionnels (bienvenue, rappels d'échéance, retards) sans les acheminer vers de véritables boîtes de réception (voir 2.4.4.4).

Infrastructure et déploiement :

- XAMPP : Distribution logicielle intégrant Apache, PHP et MySQL, utilisée pour l'environnement de développement local (voir 2.4.4.1).
- Node.js, npm et Vite : Environnement d'exécution JavaScript, gestionnaire de paquets et outil de build utilisés pour compiler les assets front-end (Tailwind CSS, Alpine.js).
- Render : Plateforme cloud (PaaS) utilisée pour l'hébergement de production de l'application et de la base de données PostgreSQL, avec pipeline de déploiement continu (voir 2.4.4.3).

2.4.2 Travaux réalisés

2.4.2.1 Gestion des ouvrages

Ce module permet aux bibliothécaires de :

- Ajouter, modifier, archiver et supprimer des ouvrages via un formulaire simple, qui enregistre les informations pertinentes dans la base de données.
- Gérer les catégories et les sous-catégories d'ouvrages.
- Gérer les exemplaires de chaque ouvrage avec leur statut (disponible, emprunté, perdu, en réparation).
- Générer automatiquement des codes-barres pour chaque exemplaire.
- Rechercher des ouvrages par titre, auteur, ISBN, catégorie.

2.4.2.2 Gestion des adhérents

Ce module permet aux bibliothécaires de :

- Inscrire de nouveaux adhérents avec saisie de leurs informations personnelles.
- Classer les adhérents par types (étudiant, enseignant, chercheur, externe).
- Générer des cartes d'adhérents en PDF avec photo et QR code intégré.
- Gérer les renouvellements d'adhésion.

- Rechercher des adhérents par nom, prénom, email ou numéro de carte.

Génération dynamique de la carte d'adhérent (PDF et QR code). Un des aspects les plus concrets du module Adhérents est la dématérialisation de la carte de membre. Dès la validation de son inscription, chaque adhérent se voit attribuer un identifiant numérique unique, encodé sous forme de QR code grâce au package simple-qrcode. Ce QR code sert de clé de reconnaissance rapide pour l'ensemble des opérations de guichet : il peut être scanné par le bibliothécaire au moment d'un emprunt ou d'un retour pour identifier instantanément l'adhérent, sans ressaisie manuelle, ce qui réduit à la fois le temps de traitement et le risque d'erreur de saisie. La carte elle-même, incluant la photo, les informations d'identité et le QR code, est générée à la volée au format PDF à l'aide de la bibliothèque dompdf, qui convertit un template HTML/CSS en document PDF conforme, sans dépendance à un outil de mise en page externe. Ce PDF est automatiquement joint à l'email de bienvenue envoyé à l'adhérent après son inscription, ce qui lui permet de disposer immédiatement de sa carte sur son téléphone ou de l'imprimer, sans avoir à se déplacer à la bibliothèque pour la récupérer. Cette approche « zéro papier » s'inscrit dans une logique de modernisation des services documentaires : elle élimine les coûts et les délais liés à l'impression physique des cartes, tout en offrant une expérience d'inscription entièrement dématérialisée, cohérente avec les usages numériques actuels.

2.4.2.3 Gestion des prêts et retours

Ce module permet aux bibliothécaires de :

- Enregistrer les prêts via scan de code-barres de l'adhérent et de l'exemplaire.
- Enregistrer les retours et mettre à jour le statut des exemplaires en temps réel.
- Vérifier automatiquement les conditions d'éligibilité (adhésion valide, quota non atteint, pas de pénalités bloquantes).
- Permettre la prolongation de prêt en ligne depuis l'espace adhérent.
- Envoyer des notifications email pour les rappels d'échéance et les retards.

2.4.2.4 Gestion des pénalités

Ce module permet aux bibliothécaires de :

- Calculer automatiquement les amendes en FCFA selon le barème configuré (nombre de jours × tarif selon type d'adhérent).
- Enregistrer les paiements partiels ou totaux des pénalités.

- Bloquer les nouveaux prêts pour les adhérents ayant des pénalités impayées.
- Générer des reçus de paiement en PDF.

2.4.2.5 Espace adhérent

Ce module permet aux usagers de :

- Consulter le catalogue public et rechercher des ouvrages.
- Visualiser leurs prêts en cours et leur historique.
- Demander une prolongation en ligne (si éligible).
- Mettre à jour leurs informations personnelles.

2.4.2.6 Tableaux de bord et statistiques

Ce module permet aux administrateurs de :

- Afficher des graphiques interactifs sur les emprunts, les ouvrages les plus consultés.
- Exporter les données en Excel et CSV.
- Générer des rapports périodiques (hebdomadaires, mensuels, annuels).

2.4.2.7 Administration

Ce module permet aux administrateurs de :

- Gérer les utilisateurs (bibliothécaires, administrateurs).
- Définir les rôles et les permissions d'accès.
- Consulter les journaux d'activité (logs) du système.

2.4.2.8 Maquettisation et design des interfaces

Avant de passer au développement des interfaces, j'ai pris le temps de concevoir des maquettes pour visualiser l'organisation de chaque écran et m'assurer que l'application serait agréable à utiliser. Cette étape m'a semblé indispensable : elle permet d'anticiper les problèmes ergonomiques avant même d'écrire la moindre ligne de code, et d'avoir une vision claire de ce qu'on veut construire.

Pour réaliser ces maquettes, j'ai utilisé Figma, un outil de prototypage en ligne très utilisé dans le domaine du développement web. Figma offre la possibilité de créer des interfaces interactives, de collaborer en temps réel et d'exporter les éléments graphiques directement utilisables dans le projet. Son interface intuitive m'a permis de travailler rapidement sans formation particulière.

Les principaux choix de design retenus sont les suivants :

- Une palette de couleurs sobre et professionnelle, avec des tons bleus et blancs pour rappeler l'univers documentaire et académique.
- Une navigation latérale fixe pour l'interface administrateur, permettant d'accéder rapidement à chaque module sans se perdre dans l'application.
- Des formulaires simples et bien espacés, pour que les bibliothécaires puissent saisir les informations rapidement, même en conditions de travail chargées.
- Un catalogue public épuré, pensé pour les adhérents : une barre de recherche centrale, des fiches d'ouvrages claires avec la disponibilité affichée en temps réel.
- Une version responsive des interfaces principales, pour que l'espace adhérent reste accessible depuis un smartphone ou une tablette.

Cette phase de maquettisation a été un vrai gain de temps par la suite. Elle m'a permis de valider la logique de navigation avant de commencer le développement, et de rester cohérent tout au long de la réalisation.

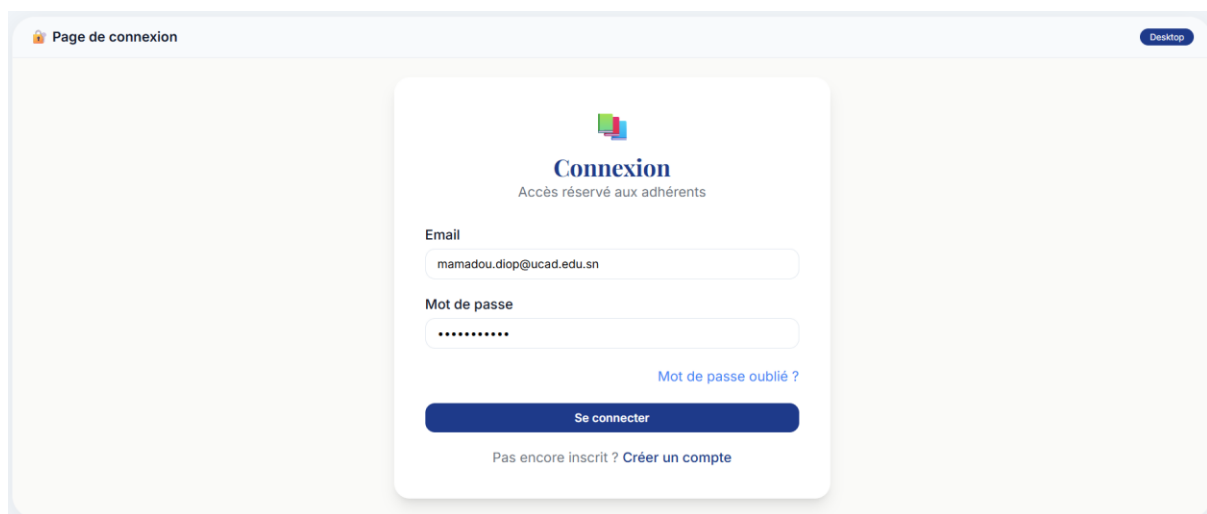


Figure 24 : Maquette Figma – Page de connexion



Figure 25 : Maquette Figma – Tableau de bord administrateur

2.4.3 Démonstration de l'application

[Figure 13 : Page d'accueil – insérer capture d'écran ici]

Figure 13 : Page d'accueil

[Figure 14 : Page de connexion – insérer capture d'écran ici]

Figure 14 : Page de connexion

[Figure 15 : Tableau de bord administrateur – insérer capture d'écran ici]

Figure 15 : Tableau de bord administrateur

[Figure 16 : Formulaire d'ajout d'un ouvrage – insérer capture d'écran ici]

Figure 16 : Formulaire d'ajout d'un ouvrage

[Figure 17 : Gestion des catégories – insérer capture d'écran ici]

Figure 17 : Gestion des catégories

[Figure 18 : Gestion des exemplaires et codes-barres – insérer capture d'écran ici]

Figure 18 : Gestion des exemplaires et codes-barres

[Figure 19 : Liste des ouvrages – insérer capture d'écran ici]

Figure 19 : Liste des ouvrages

[Figure 20 : Gestion des adhérents – insérer capture d'écran ici]

Figure 20 : Gestion des adhérents

[Figure 21 : Carte adhérent PDF – insérer capture d'écran ici]

Figure 21 : Carte adhérent PDF

[Figure 22 : Gestion des prêts en cours – insérer capture d'écran ici]

Figure 22 : Gestion des prêts en cours

[Figure 23 : Gestion des pénalités – insérer capture d'écran ici]

Figure 23 : Gestion des pénalités

2.4.4 Déploiement de la solution

Une fois le développement terminé et les principaux modules testés, j'ai procédé au déploiement de l'application dans un environnement local stable. Cette étape conclut le cycle de développement et permet de s'assurer que l'application fonctionne correctement en dehors de l'environnement de développement.

2.4.4.1 Environnement de déploiement

Le déploiement a été réalisé sur un serveur local configuré avec les composants suivants :

- Serveur web Apache, intégré via XAMPP, pour servir l'application Laravel.
- PHP 8.2 comme environnement d'exécution côté serveur.
- MySQL pour la base de données en local (PostgreSQL en production sur Render, voir 2.4.4.3), avec les tables générées via les migrations Laravel.
- Node.js et npm pour la compilation des assets front-end avec Vite.

Toute la configuration de l'application est centralisée dans le fichier .env, ce qui facilite la portabilité du projet : il suffit d'adapter les variables d'environnement pour passer d'un poste à un autre ou d'un serveur local à un hébergement en ligne.

2.4.4.2 Étapes de mise en production locale

Les étapes que j'ai suivies pour déployer l'application sont les suivantes :

- Clonage du dépôt GitHub sur la machine cible avec la commande git clone.
- Installation des dépendances PHP via composer install et des dépendances JavaScript via npm install.
- Configuration du fichier .env : connexion à la base de données, clé d'application, paramètres mail.
- Génération de la clé d'application avec php artisan key:generate.

- Exécution des migrations et des seeders pour initialiser la base de données : `php artisan migrate --seed`.
- Compilation des assets avec `npm run build` pour générer les fichiers CSS et JS optimisés.
- Lancement du serveur de développement avec `php artisan serve` pour vérifier le bon fonctionnement de l'ensemble.

Au-delà de l'environnement local, BiblioSmart a été effectivement déployée en production sur la plateforme cloud Render, avec une base de données PostgreSQL, la structure Laravel ayant facilité cette transition : le code est propre, bien organisé, et les dépendances sont toutes déclarées dans les fichiers de configuration. Les sections suivantes détaillent deux aspects de cette mise en production qui ont demandé une attention particulière : l'automatisation du déploiement par un pipeline CI/CD, et la sécurisation des envois d'emails transactionnels.

2.4.4.3 Intégration et déploiement continu (CI/CD)

Plutôt que de déployer manuellement chaque évolution de l'application, un pipeline d'intégration et de déploiement continu (CI/CD) a été mis en place en reliant le dépôt GitHub du projet à Render via un fichier de configuration `render.yaml`. Ce fichier fonctionne selon le principe de l'Infrastructure as Code : il décrit, dans un format déclaratif versionné avec le reste du projet, le type de service (application web PHP), la commande de build, les variables d'environnement nécessaires et la base de données PostgreSQL associée. Concrètement, chaque envoi de code (git push) sur la branche principale déclenche automatiquement une nouvelle exécution du pipeline sur Render, qui installe les dépendances via composer, exécute les migrations de base de données, lance les seeders si nécessaire, puis optimise l'application (mise en cache des routes, de la configuration et des vues) avant de basculer le trafic vers la nouvelle version. Render gère ce basculement de façon à limiter au maximum l'interruption de service, ce qui garantit une disponibilité quasi continue de l'application, y compris pendant les phases de mise à jour. La mise en place de ce pipeline a représenté un gain de temps significatif en phase de développement itératif et constitue une pratique standard des équipes de développement professionnelles.

2.4.4.4 Sécurité et délivrabilité des emails transactionnels

L'application envoie plusieurs types d'emails automatiques : email de bienvenue avec la carte d'adhérent en pièce jointe, rappels d'échéance de prêt, notifications de retard. La mise en production de ce module a soulevé une difficulté qui dépasse le cadre strictement applicatif :

celle de la délivrabilité des emails. En effet, les fournisseurs de messagerie s'appuient aujourd'hui sur trois protocoles de sécurité complémentaires pour lutter contre l'usurpation d'identité et le phishing : SPF (Sender Policy Framework), qui définit la liste des serveurs autorisés à envoyer des emails pour un domaine donné ; DKIM (DomainKeys Identified Mail), qui signe cryptographiquement chaque message pour en garantir l'authenticité et l'intégrité ; et DMARC (Domain-based Message Authentication, Reporting and Conformance), qui définit la politique à appliquer lorsque les vérifications SPF ou DKIM échouent (mise en quarantaine ou rejet du message). Ces mécanismes ont pour conséquence directe qu'une adresse d'expédition générique de type @gmail.com ne peut pas être utilisée telle quelle par une application tierce pour envoyer des emails en masse en production : Gmail exige que l'expéditeur prouve son autorité sur le domaine utilisé, ce qu'une adresse personnelle ne permet pas. Pour contourner cette contrainte tout en conservant un flux de test fiable pendant la phase de développement, l'application utilise Mailtrap, une solution de messagerie « sandbox » reconnue comme standard de l'industrie : Mailtrap intercepte les emails envoyés par l'application dans un environnement isolé, sans jamais les acheminer vers de véritables boîtes de réception, ce qui permet de vérifier le contenu, la mise en forme et les pièces jointes de chaque notification sans risquer d'impact sur des adresses réelles ni de se heurter aux restrictions imposées par les fournisseurs de messagerie grand public. Cette contrainte, rencontrée concrètement lors du développement, a permis de mieux appréhender les exigences réelles de la délivrabilité des emails en environnement de production.

2.4.5 Nouvelles fonctionnalités développées : vers un produit professionnel

Au-delà des fonctionnalités fonctionnelles de base couvertes par les sept modules initiaux, une seconde itération du projet a été consacrée à faire évoluer BiblioSmart d'un prototype académique vers un produit présentant les standards d'ergonomie et d'accessibilité attendus d'une application professionnelle utilisée sur le terrain. Cinq axes d'amélioration ont été retenus, détaillés ci-après.

2.4.5.1 Transformation en Progressive Web App (PWA)

L'application a été transformée en Progressive Web App (PWA) installable, grâce à l'implémentation d'un fichier de manifeste (manifest.json) décrivant l'icône, le nom et le thème de l'application, ainsi que d'un Service Worker (sw.js) chargé de la mise en cache des ressources statiques. Cette évolution permet aux bibliothécaires comme aux adhérents d'installer BiblioSmart directement sur l'écran d'accueil de leur smartphone ou de leur tablette, sous

Android comme sous iOS, sans passer par un store d'applications. Une fois installée, l'application s'ouvre en plein écran, sans barre d'adresse du navigateur, offrant une expérience immersive proche d'une application native tout en conservant l'unicité du code base web.

2.4.5.2 Interface responsive et mobile-first pour l'administration

L'interface d'administration, initialement pensée pour un usage sur poste fixe, a été retravaillée pour un usage sur le terrain via tablette ou smartphone. Trois évolutions structurent cette refonte :

- Une barre de navigation inférieure (bottom navigation bar), pattern ergonomique standard sur mobile, remplace la navigation latérale classique sur les petits écrans et permet d'accéder aux modules principaux d'un simple geste du pouce.
- Les tableaux de données complexes (listes d'ouvrages, d'adhérents, de prêts) sont automatiquement transformés en un affichage sous forme de cartes (card-based layout) sur mobile, ce qui supprime le défilement horizontal inconfortable propre aux tableaux HTML classiques sur petit écran.
- Un menu latéral escamotable, piloté par Alpine.js, permet de replier la navigation sur les écrans intermédiaires afin de maximiser l'espace disponible pour les zones de travail (formulaires, tableaux de bord).

2.4.5.3 Scanner de QR code intégré pour les emprunts et retours

Afin de s'affranchir de la dépendance à un lecteur de codes-barres USB physique, la librairie html5-qrcode a été intégrée pour accéder directement à la caméra de l'appareil de l'utilisateur depuis le navigateur. Depuis le formulaire de prêt, le bibliothécaire peut désormais déclencher le scanner et utiliser la caméra de son téléphone ou de sa tablette pour lire la carte de l'adhérent, physique ou numérique. Le code capturé est transmis en temps réel au composant Livewire 3 côté serveur, qui interroge la base de données et sélectionne automatiquement l'adhérent correspondant dans le formulaire, sans ressaisie manuelle ni rechargement de page.

2.4.5.4 Refonte de la carte adhérent numérique

La carte adhérent générée dynamiquement en PDF via dompdf (voir 2.4.2.2) a fait l'objet d'une seconde itération visuelle, motivée par des retours d'usage concrets :

- Au recto, la typographie (interlignage et espacements) a été retravaillée pour éviter les chevauchements de texte constatés avec les noms et prénoms longs.

- Au verso, le code-barres unidimensionnel initial a été remplacé par un QR code de grande taille et centré, généré algorithmiquement sans marges parasites, garantissant une lisibilité et une scannabilité fiables même avec des caméras de téléphone d'entrée de gamme.
- Le QR code est également affiché en accès direct sur le profil numérique de l'adhérent dans l'interface, ce qui permet un scan rapide au guichet sans devoir télécharger le fichier PDF au préalable.

2.4.5.5 Architecture frontend réactive (Livewire 3 + Alpine.js)

L'ensemble de ces nouvelles fonctionnalités repose sur l'intégration native offerte par Livewire 3 avec Alpine.js. Cette combinaison permet de piloter les modales, les menus escamotables et le flux vidéo du scanner de QR code sans rechargement de page, en confiant à Alpine.js la gestion des micro-interactions côté client (ouverture/fermeture d'un composant, transitions) pendant que Livewire conserve la maîtrise de la logique métier côté serveur. Le résultat est une fluidité d'utilisation comparable à celle d'une Single Page Application, sans les coûts de développement et de maintenance associés à un framework JavaScript dédié.

TROISIEME CHAPITRE III

BILAN

Bilan

Objectif n°1 : Gestion des ouvrages

Atteint : La mise en place d'un système complet de catalogage et de gestion des exemplaires a été réalisée avec succès. Ce module permet l'ajout, la modification, l'archivage et la suppression d'ouvrages, la gestion des catégories et des sous-catégories, ainsi que la génération automatique de codes-barres pour chaque exemplaire. Cela a assuré une gestion efficace du fonds documentaire tout en garantissant la traçabilité des ouvrages.

Objectif n°2 : Gestion des adhérents

Atteint : Le module de gestion des adhérents permet désormais aux bibliothécaires d'inscrire de nouveaux adhérents, de gérer les renouvellements d'adhésion et de générer des cartes d'adhérents en PDF avec QR code intégré. La classification par types (étudiant, enseignant, chercheur, externe) permet une gestion personnalisée des quotas de prêts.

Objectif n°3 : Gestion des prêts et retours

Atteint : Le système de gestion des prêts et retours a été entièrement développé. Il permet l'enregistrement des prêts et retours via scan de code-barres, la vérification automatique des conditions d'éligibilité, la prolongation de prêt en ligne, et l'envoi de notifications email pour les rappels d'échéance et les retards.

Objectif n°4 : Gestion des pénalités

Atteint : La fonctionnalité de gestion des pénalités a été entièrement développée pour calculer automatiquement les amendes en FCFA selon le barème configuré, enregistrer les paiements partiels ou totaux, et bloquer les nouveaux prêts pour les adhérents ayant des pénalités impayées. Cela garantit un recouvrement efficace des amendes.

Objectif n°5 : Espace adhérent

Atteint : L'espace adhérent permet aux usagers de consulter le catalogue public, de visualiser leurs prêts en cours et leur historique, de demander une prolongation en ligne et de mettre à jour leurs informations personnelles. Cela améliore l'expérience utilisateur et réduit les déplacements.

Objectif n°6 : Tableaux de bord et statistiques

Partiellement atteint : Ce module est en cours de développement. Les graphiques interactifs de base ont été implémentés, mais des fonctionnalités plus avancées (exports avancés, rapports personnalisés) restent à compléter.

Objectif n°7 : Administration

Atteint : Le module d'administration permet de gérer les utilisateurs, de définir les rôles et les permissions d'accès, et de consulter les journaux d'activité (logs) du système.

Intérêts personnels

Ce projet m'a permis d'acquérir des compétences cruciales en développement logiciel, en particulier dans la conception de modules fonctionnels pour la gestion d'ouvrages, d'adhérents, de prêts et de pénalités. J'ai amélioré ma maîtrise des technologies web modernes comme Laravel pour le backend et Blade + Tailwind CSS + Livewire pour le frontend.

J'ai également développé une grande rigueur en gestion de projet, en m'adaptant aux méthodes Agile/Scrum et en utilisant des outils professionnels comme Trello (tableau Kanban) et GitHub (versionnement). La conception de ce projet m'a permis d'approfondir ma compréhension des besoins fonctionnels liés à la gestion documentaire et de traduire ces besoins en solutions techniques concrètes.

Enfin, cette expérience m'a aidé à comprendre l'importance de la sécurité et de l'optimisation des performances dans la gestion de données sensibles. Elle constitue un tremplin important pour la suite de ma carrière dans le développement logiciel.

Apport du projet pour les bibliothèques

Au-delà de ce que ce projet m'a personnellement apporté, il me semble important de réfléchir à ce qu'une telle application peut réellement changer pour une bibliothèque qui l'adopterait. C'est finalement l'utilité concrète du travail réalisé.

Le premier apport évident est le gain de temps. Aujourd'hui, dans beaucoup de bibliothèques, les bibliothécaires passent une bonne partie de leur journée à gérer des registres, à chercher des informations dans des fichiers éparpillés, à calculer manuellement des pénalités. Avec cette application, ces tâches deviennent automatiques. Le bibliothécaire peut se concentrer sur l'essentiel : accueillir les usagers, conseiller, animer.

Pour les adhérents, l'application change aussi beaucoup de choses. Pouvoir consulter le catalogue depuis chez soi, savoir si un ouvrage est disponible avant de se déplacer, prolonger un prêt en ligne sans avoir à revenir physiquement — ce sont des services simples mais qui améliorent vraiment l'expérience au quotidien.

Du côté de la gestion, les tableaux de bord et les statistiques offrent aux responsables une vue d'ensemble qu'ils n'avaient pas avant. Quels sont les ouvrages les plus empruntés ? Quels adhérents ont des pénalités en cours ? Combien de prêts ont été effectués ce mois-ci ? Ces données, disponibles en quelques clics, permettent de prendre de meilleures décisions : acheter les ouvrages qui manquent, relancer les adhérents inactifs, ajuster les barèmes.

Enfin, la solution est conçue pour être adaptable. Qu'il s'agisse d'une petite bibliothèque scolaire, d'une médiathèque municipale ou d'une bibliothèque universitaire, les fonctionnalités restent les mêmes. Seuls les paramétrages changent. C'est là, je crois, la vraie valeur de ce projet : proposer un outil générique, robuste et maintenable, que n'importe quel établissement pourrait déployer et s'approprier.

BIBLIOGRAPHIE

Ouvrages :

- UML en action (de l'analyse des besoins à la conception), Pascal Roques et Franck Vallée, 4e édition Eyrolles.
- Chaumier, J. (2002). Travail et méthodes du documentaliste : Connaissance du problème. ESF Éditeur.
- Otlet, P. (1934). Traité de documentation : Le livre sur le livre. Mundaneum.
- Stauffer, M. (2019). Laravel: Up and Running – A Framework for Building Modern PHP Apps. O'Reilly Media.
- Lemay, L., & Cadenhead, R. (2010). Sams Teach Yourself PHP, MySQL and Apache. Sams Publishing.

WEBOGRAPHIE

- Installation - Laravel 11.x - The PHP Framework For Web Artisans : 10/07/2024, 00h10
- Livewire - Full-stack framework for Laravel : 10/07/2024, 00h30
- Authentication - Laravel 11.x - The PHP Framework For Web Artisans : 15/08/2024, 23h12
- Git Guides - install git (github.com) : 20/08/2024, 21h24
- Tailwind CSS - Documentation officielle : 05/09/2024, 02h12
- Bibliothèque nationale de France – À propos des bibliothèques : bnf.fr : 07/09/2024, 00h18
- Qu'est-ce que l'architecture à trois niveaux | IBM : 05/08/2024, 00h18
- Alpine.js – Documentation officielle : 12/09/2024, 10h00

Table des matières

A la mémoire de	I
Dédicace	II
Remerciements	III
Avant-propos	IV
Glossaire	VI
Liste des figures	VII
Résumé	VIII
Abstract	IX
PREMIER CHAPITRE I – INTRODUCTION GÉNÉRALE	1
1.1 Présentation du domaine des bibliothèques	2
1.2 Contexte	3
1.3 Problématique	4
1.4 Objectifs	5
DEUXIEME CHAPITRE II – Etude et Réalisation	6
2.1 Gestion de projet agile	7
2.1.1 Trello	7
2.2 Spécification fonctionnelle	8
2.2.1 Les besoins fonctionnels	8
2.2.2 Les besoins non-fonctionnels	9
2.3 Analyse et conception	11
2.3.1 La méthode UML	11
2.3.2 Le diagramme de contexte	11
2.3.3 Le diagramme de cas d'utilisation	12
2.3.4 Le diagramme de classe	14

2.4 Réalisation	14
2.4.1 Outils et technologies utilisés	14
2.4.1.1 Architecture	14
2.4.1.2 Technologies et langages	15
2.4.1.3 Outils	17
2.4.2 Travaux réalisés	18
2.4.2.1 Gestion des ouvrages	18
2.4.2.2 Gestion des adhérents	18
2.4.2.3 Gestion des prêts et retours	19
2.4.2.4 Gestion des pénalités	19
2.4.2.5 Espace adhérent	19
2.4.2.6 Tableaux de bord et statistiques	19
2.4.2.7 Administration	19
2.4.2.8 Maquettisation et design des interfaces	22
2.4.3 Démonstration de l'application	23
2.4.4 Déploiement de la solution	30
 TROISIEME CHAPITRE III – Bilan	 25
Bilan	25
Intérêts personnels	26
 BIBLIOGRAPHIE	 i
WEBOGRAPHIE	ii
Table des matières	iii